

M7 - Introduction to Code Analysis

Guillermo Polito



Inria



**Université
de Lille**

Goals

- Code Analysis
- Control flow graphs
- Single Static Assignment and Phi functions
- Data dependencies
- Dataflow analysis

Terminology

Inter vs Intra Procedural

- **Inter-procedural** analyses:
 - analyses *between* procedures (*i.e.*, functions, methods, choose yours)
- **Intra-procedural** analyses
 - analyses *within* procedures

Global vs Local Analyses

- **Global** analyses:
 - Take a *full* procedure into account
- **Local** analyses:
 - analyses *within* a little part of a procedure (a basic block, wait a bit)

Guiding Example 1: linear code

```
def f(param){  
    local y := param + 7.  
    local z := y + 1.  
    local h := z + 1.  
    h := h + z.  
    y := y + 1.  
    return y  
}
```


Objective: dead code detection

```
def f(param){  
  local y := param + 7.  
  local z := y + 1.  
  local h := z + 1.  
  h := h + z.  
  y := y + 1.  
  return y  
}
```

- *h* and *z* are clearly dead code
- *z* is indirectly not used
- *h* uses itself
- How do we automatically detect them?

Dataflow Basics: instruction dependencies

```
def f(param){  
  local y := param + 7.  
  local z := y + 1.  
  local h := z + 1.  
  h := h + z.  
  y := y + 1.  
  return y  
}
```

The diagram illustrates dataflow dependencies in the provided code. Solid arrows represent dependencies that are used in the program: an arrow from 'param' to 'y' in the first line, from 'y' to 'z' in the second line, from 'z' to 'h' in the third line, from 'h' to 'h' in the fourth line, and from 'y' to 'return y' in the sixth line. Dashed arrows represent dependencies that are not used: an arrow from 'z' to 'h' in the third line and from 'h' to 'h' in the fourth line.

- The return value does not depend on dead code
- Each variable *use* depends on its previous *write*
- In dataflow terms we have
 - uses
 - defs (definitions, on every write)

Fixpoint: Iterative Dataflow Analysis

- Looking for *convergence (fix point)*
- Each iteration uses the facts discovered by previous iterations
- Stops when I do not find anything new

```
def f(param){  
    changed := false;  
    while (!changed) {  
        changed := analyse();  
    }  
}
```

Example: Iterative Dead Code Analysis

- Start with everything dead

```
def f(param){  
  Dead local y := param + 7.  
  
  Dead local z := y + 1.  
  
  Dead local h := z + 1.  
  
  Dead h := h + z.  
  
  Dead y := y + 1.  
  
  Dead return y  
}
```

Example: Iterative Dead Code Analysis

- Start with everything dead
- Mark as Live everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*

```
def f(param){  
  Dead local y := param + 7.  
  
  Dead local z := y + 1.  
  
  Dead local h := z + 1.  
  
  Dead h := h + z.  
  
  Dead y := y + 1.  
  
  Live return y  
}
```


Example: Iterative Dead Code Analysis

- Start with everything dead
- Mark as Live everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*
- Mark as Live all dependencies of Live entries

```
def f(param){  
  Dead local y := param + 7.  
  Dead local z := y + 1.  
  Dead local h := z + 1.  
  Dead h := h + z.  
  Live y := y + 1.  
  Live return y  
}
```

Example: Iterative Dead Code Analysis

- Start with everything dead
- Mark as Live everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*
- Mark as Live all dependencies of Live entries
- Iterate

```
def f(param){  
  Live  local y := param + 7.  
  Dead  local z := y + 1.  
  Dead  local h := z + 1.  
  Dead  h := h + z.  
  Live  y := y + 1.  
  Live  return y  
}
```

Example: Iterative Dead Code Analysis

- Start with everything dead
- Mark as Live everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*
- Mark as Live all dependencies of Live entries
- Iterate
- Iterate, nothing changed, stop

```
def f(param){  
  Live  local y := param + 7.  
  Dead  local z := y + 1.  
  Dead  local h := z + 1.  
  Dead  h := h + z.  
  Live  y := y + 1.  
  Live  return y  
}
```


Iterative Analyses

- Simple to implement
- Can be costly if too much to analyse
 - Looks at the entire code each iteration

```
def f(param){  
    changed := false;  
    while (!changed) {  
        changed := analyse();  
    }  
}
```

Improving with Worklist Algorithms

- Looking for *convergence* (*fix point*)
- Code to evaluate is put on a queue (the worklist)
- Each iteration consumes an element from the worklist
- Analysis adds back in the worklist new things to evaluate

```
def f(initialSet){
    worklist := initialSet;
    while (notEmpty(worklist)) {
        analyse(pop(worklist));
    }
}

def analyse(task){
    ...
    addAll(worklist, dependencies(task));
}
```


Example: Worklist Dead Code Analysis

- Put in the worklist everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*

```
def f(param){  
S1  local y := param + 7.  
S2  local z := y + 1.  
S3  local h := z + 1.  
S4  h := h + z.  
S5  y := y + 1.  
S6  return y  
}
```

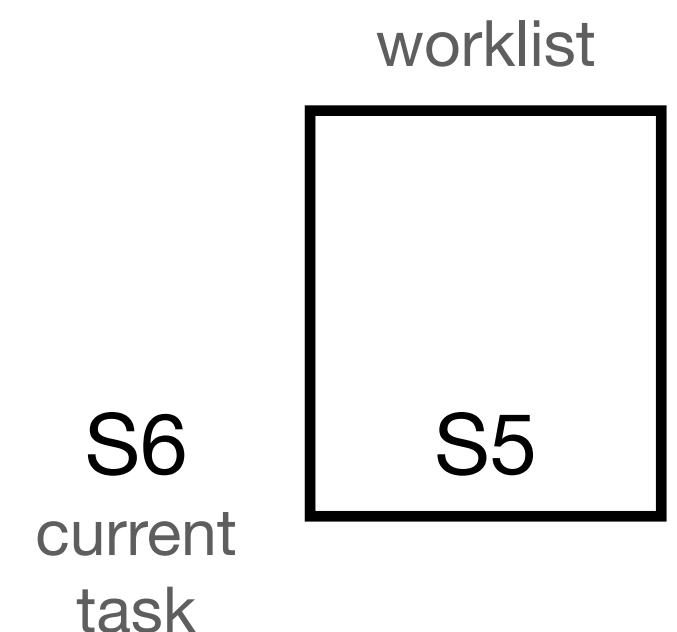
worklist

S6

Example: Worklist Dead Code Analysis

- Put in the worklist everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*
- Take an element from the worklist, mark it as Live and add dependencies to worklist
- Iterate

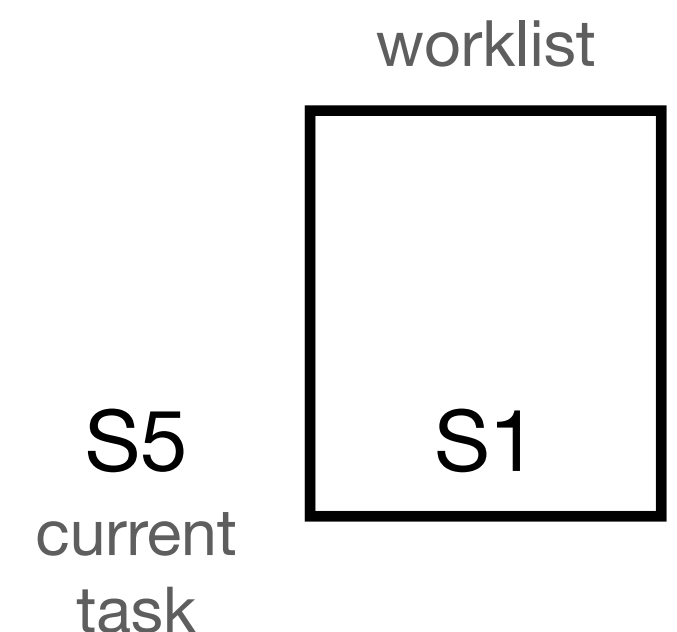
```
def f(param){  
S1  local y := param + 7.  
S2  local z := y + 1.  
S3  local h := z + 1.  
S4  h := h + z.  
S5  y := y + 1.  
Live return y  
}
```



Example: Worklist Dead Code Analysis

- Put in the worklist everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*
- Take an element from the worklist, mark it as Live and add dependencies to worklist
- Iterate

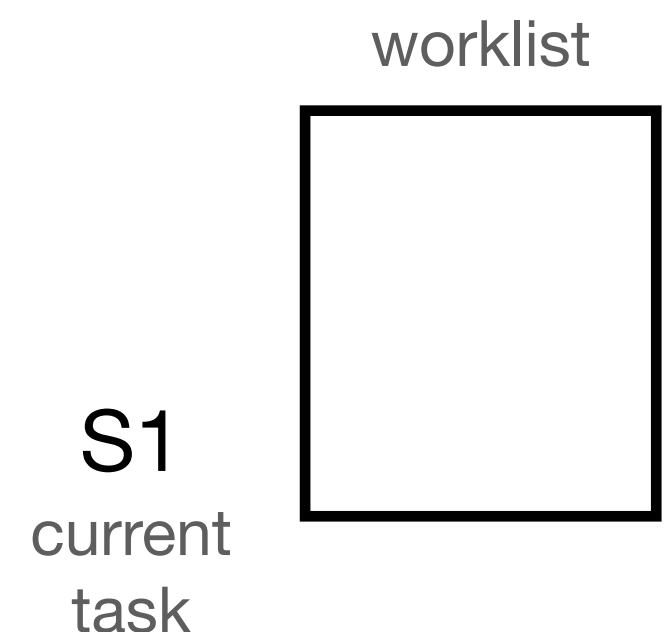
```
def f(param){  
  S1  local y := param + 7.  
  S2  local z := y + 1.  
  S3  local h := z + 1.  
  S4  h := h + z.  
  Live y := y + 1.  
  Live return y  
}
```



Example: Worklist Dead Code Analysis

- Put in the worklist everything that is observable from the outside: *i.e.*, *returns, side-effects, function calls*
- Take an element from the worklist, mark it as Live and add dependencies to worklist
- Iterate
- Iterate, worklist empty, stop

```
def f(param){  
  Live  local y := param + 7.  
  S2    local z := y + 1.  
  S3    local h := z + 1.  
  S4    h := h + z.  
  Live  y := y + 1.  
  Live  return y  
}
```



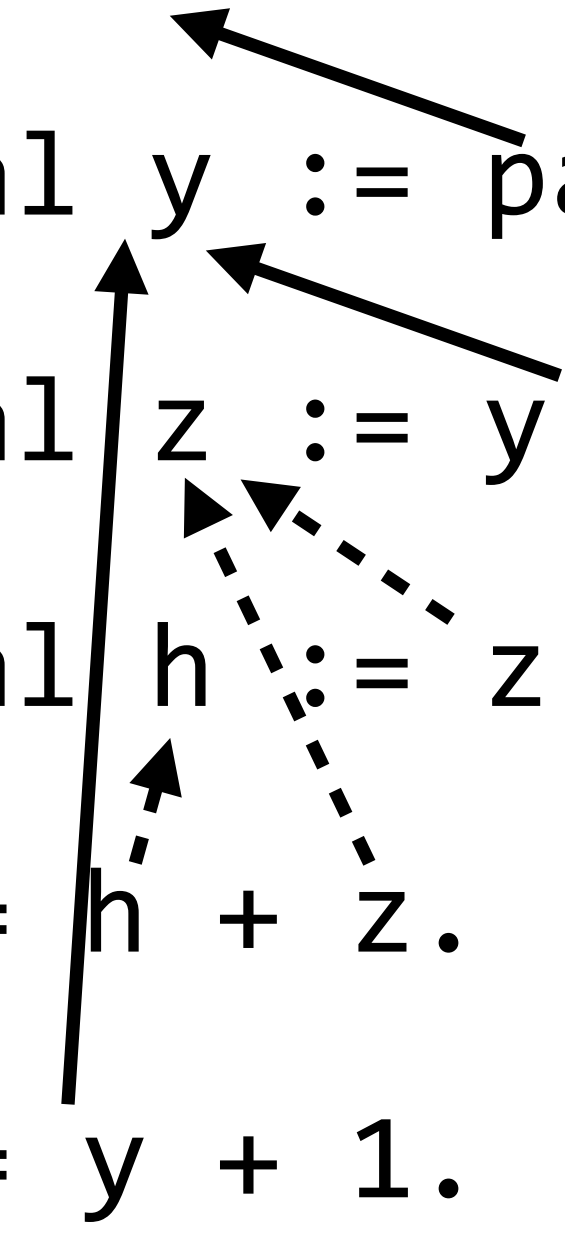
Worklist Algorithms

- Also simple to implement
- Less costly
 - Looks only at what is in the worklist

```
def f(initialSet){  
    worklist := initialSet;  
    while (notEmpty(worklist)) {  
        analyse(pop(worklist));  
    }  
}  
  
def analyse(task){  
    ...  
    addAll(worklist, dependencies(task));  
}
```

Back to Example 1: Computing Dependencies

```
def f(param){  
s1  local y := param + 7.  
s2  local z := y + 1.  
s3  local h := z + 1.  
s4  h := h + z.  
s5  y := y + 1.  
s6  return y  
}
```



- We can compute uses/defs as a dataflow problem
- Idea: Record at each point the possible values
 - E.g., at S2, the valid definition of y is S1's y
 - S5 overrides this definition.
 - Thus, at S6, the valid definition of y is S5's y
- Why dataflow? because with control flow this can get messy

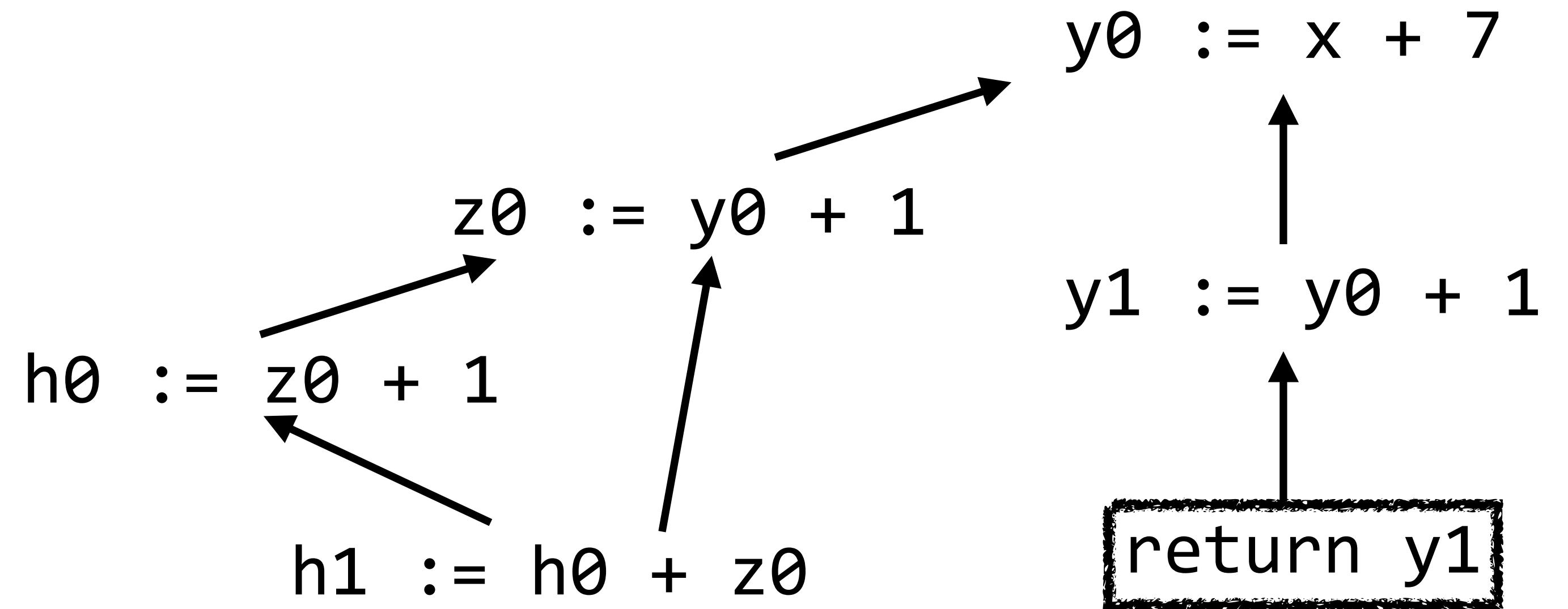
Single Static Assignment: explicit dependencies

```
def f(param){  
  local  $y_0$  := param + 7.  
  local  $z_0$  :=  $y_0$  + 1.  
  local  $h_0$  :=  $z_0$  + 1.  
  local  $h_1$  :=  $h_0$  +  $z_0$ .  
  local  $y_1$  :=  $y_0$  + 1.  
  return  $y_1$   
}
```

- Each local assignment is a *definition*
- **Stronger:** No variable has more than one assignment
- Each new variable is a *version* of the original
- We can at a “sight” track dependencies
 - Where is h_0 defined? Clear!
 - Is h_1 used? Nope! Transitively, h_0 is not used either

SSA as Explicit Data Dependency Graph

- Modeling as a graph
- Edges are dependencies



- Disconnected clusters => dead code

Guiding Example 2: conditionals

```
def f(param){  
  local y.  
  param > 0 ifTrue: [  
    y := 17  
  ] ifFalse: [  
    y := 42  
  ].  
  return y  
}
```

Conditionals in SSA

```
def f(param){  
  param > 0 ifTrue: [  
    local  $y_0$  := 17  
  ] ifFalse: [  
    local  $y_1$  := 42  
  ].  
  return  $y$ ?  
}
```

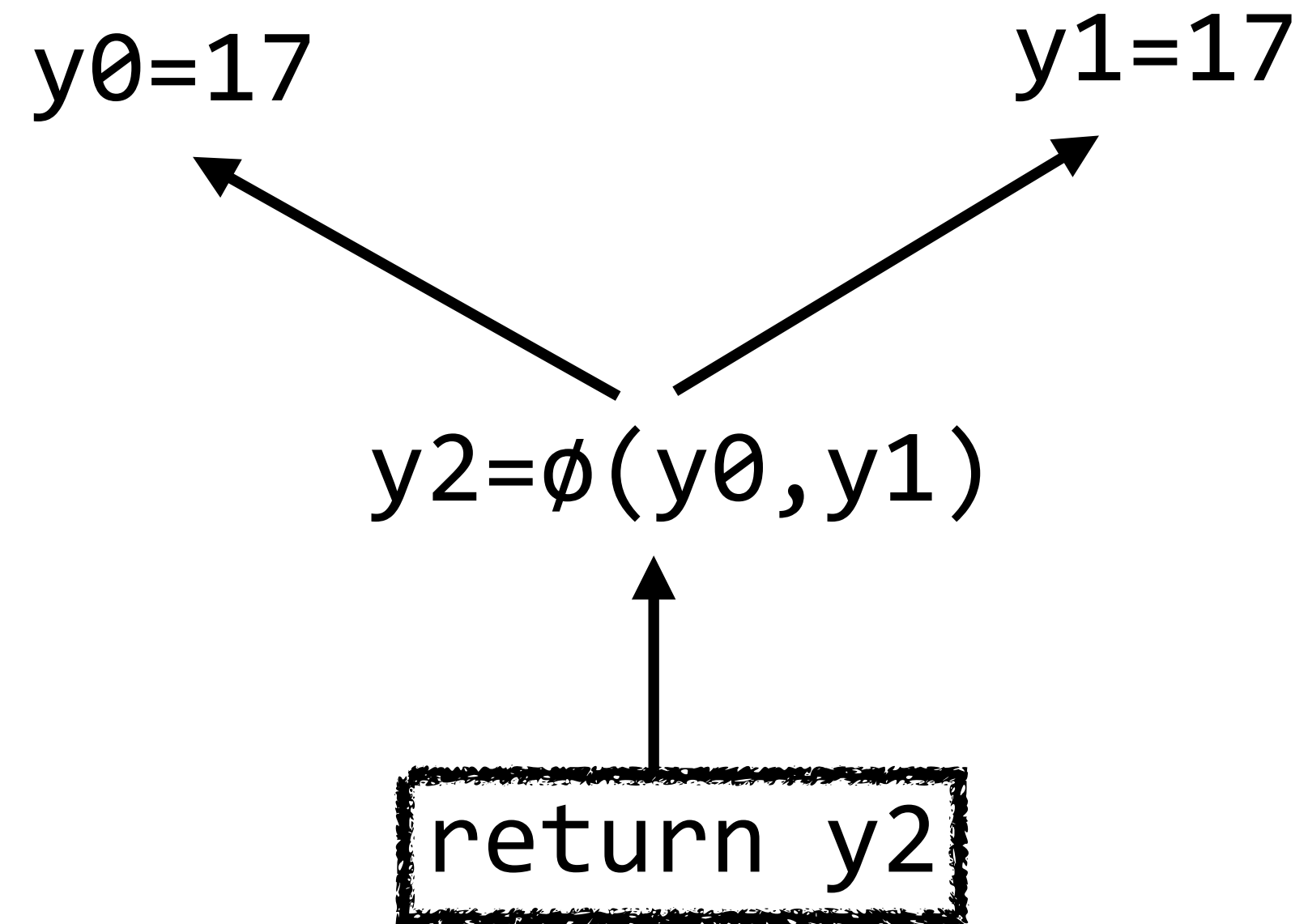
- What version should we return?
 - y_0 if we come from the *true branch*
 - y_1 if we come from the *false branch*

Phi functions

```
def f(param){  
    param > 0 ifTrue: [  
        local y0 := 17  
    ] ifFalse: [  
        local y1 := 42  
    ].  
    local y2 :=  $\phi(y_0, y_1)$   
    return y2  
}
```

- Phi functions are *virtual instructions* in join points
 - maintain the SSA property
 - maintain dependencies
- in a compiler, they do not have a translation

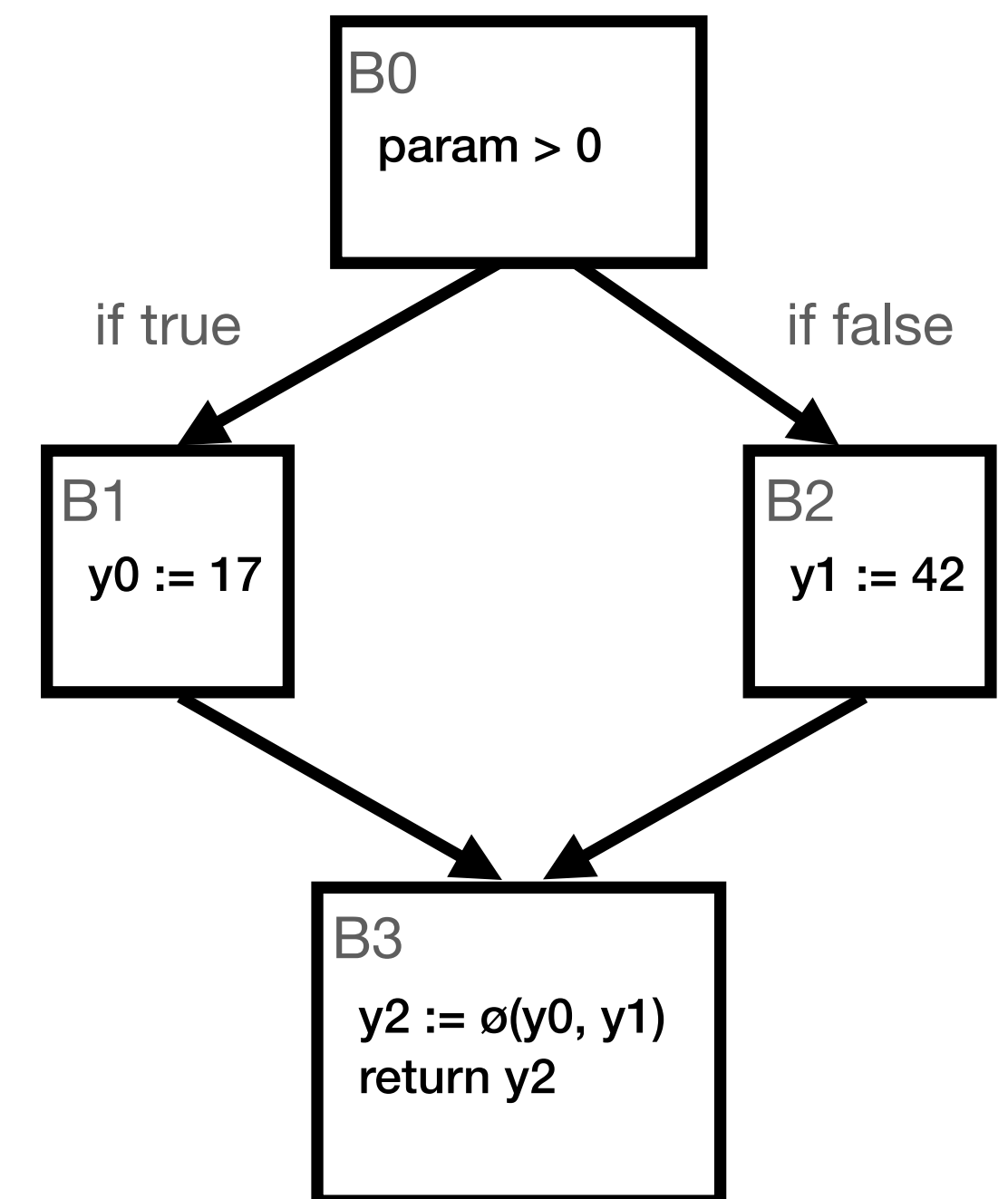
Phi functions dependency graph



- Where is the control flow?
- We lost the conditional!

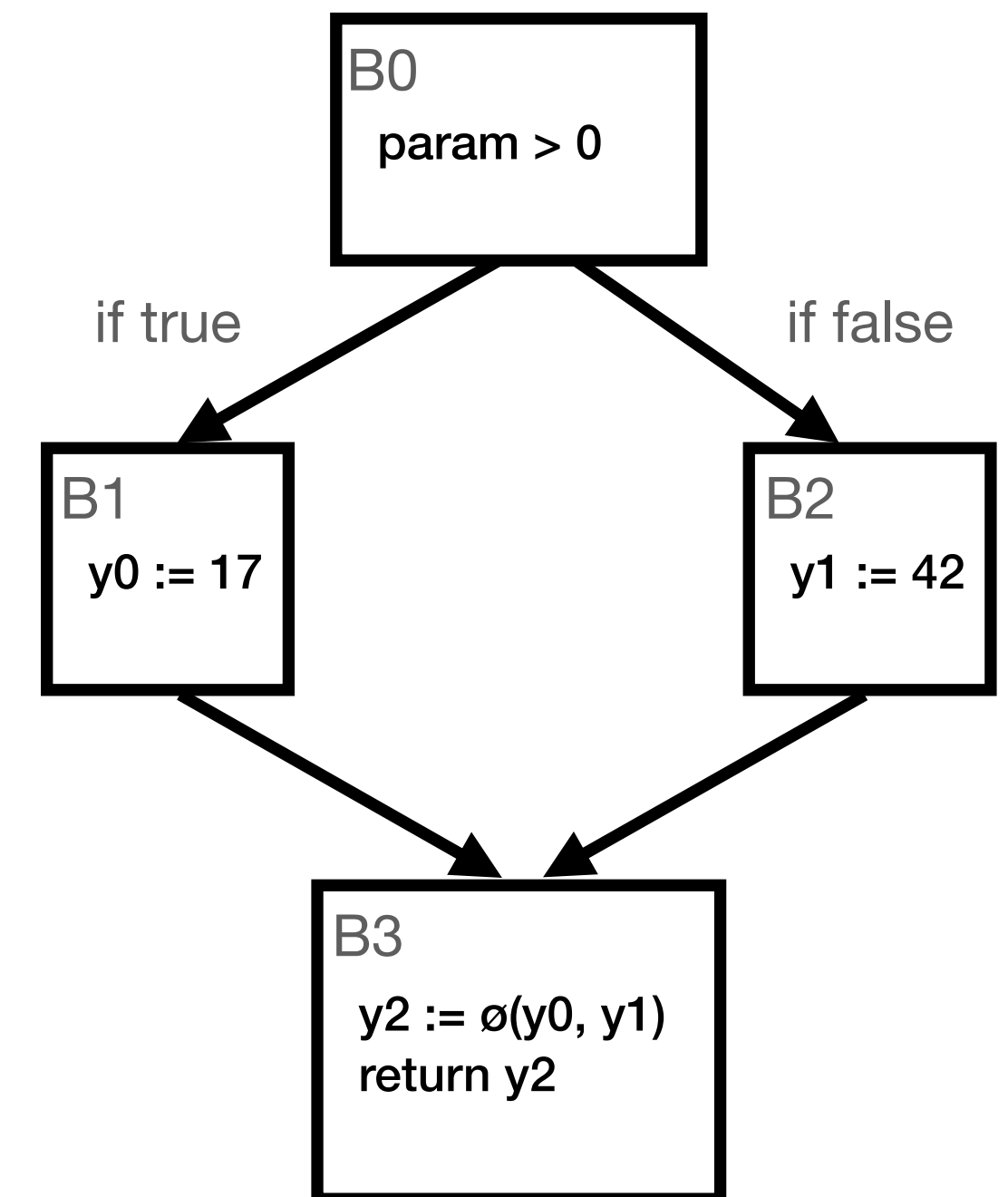
Control-flow graphs example

```
def f(param){  
  param > 0 ifTrue: [  
    local y0 := 17  
  ] ifFalse: [  
    local y1 := 42  
  ].  
  local y2 :=  $\phi(y_0, y_1)$   
  return y2  
}
```



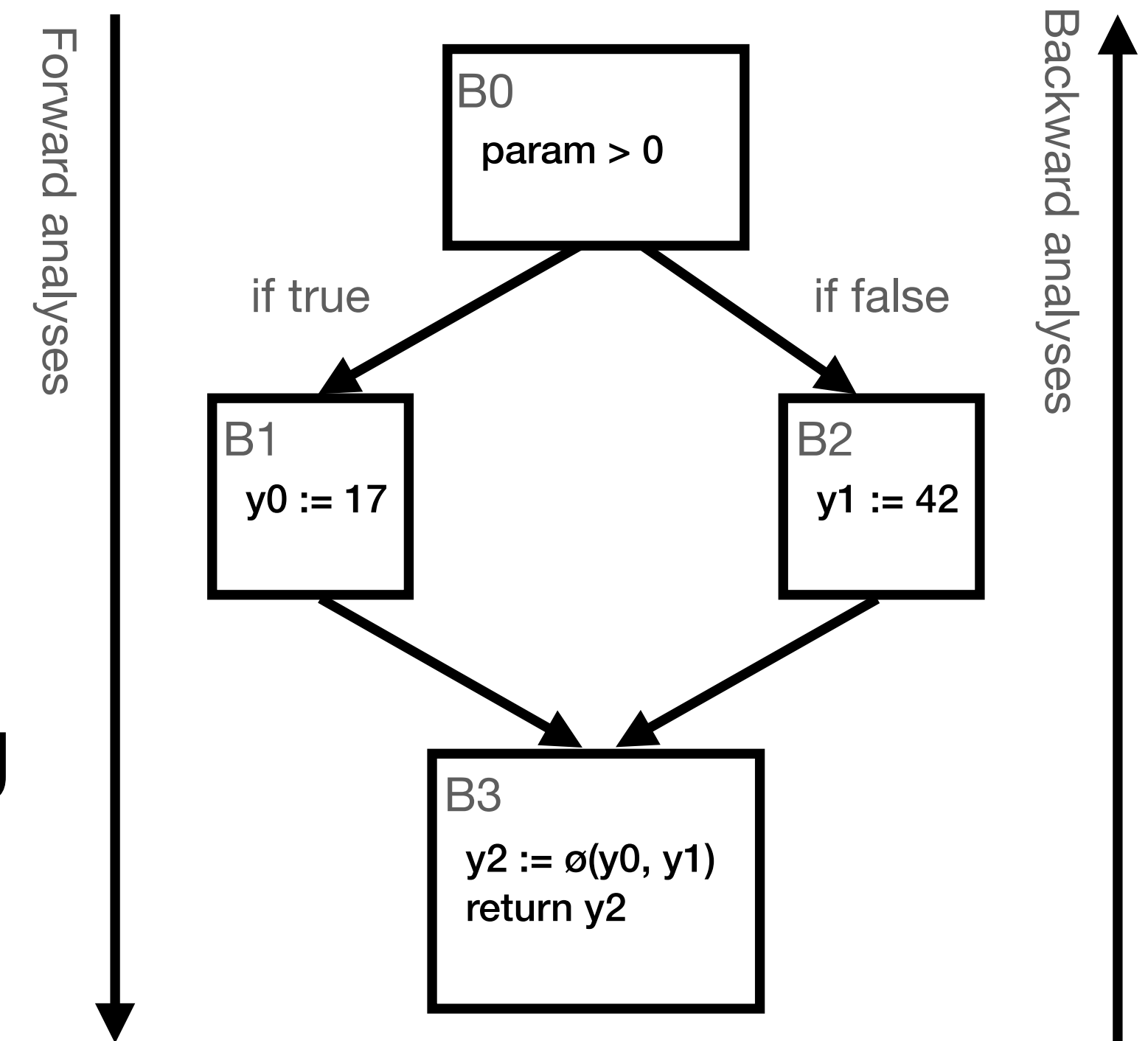
Control-flow graphs explained

- Graph made of *basic blocks*
- Each basic block represents a linear sequence of instructions
- Block edges: predecessors and successors
 - *Conditional*: Block with multiple successors
 - *Merge Point*: Block with multiple predecessors

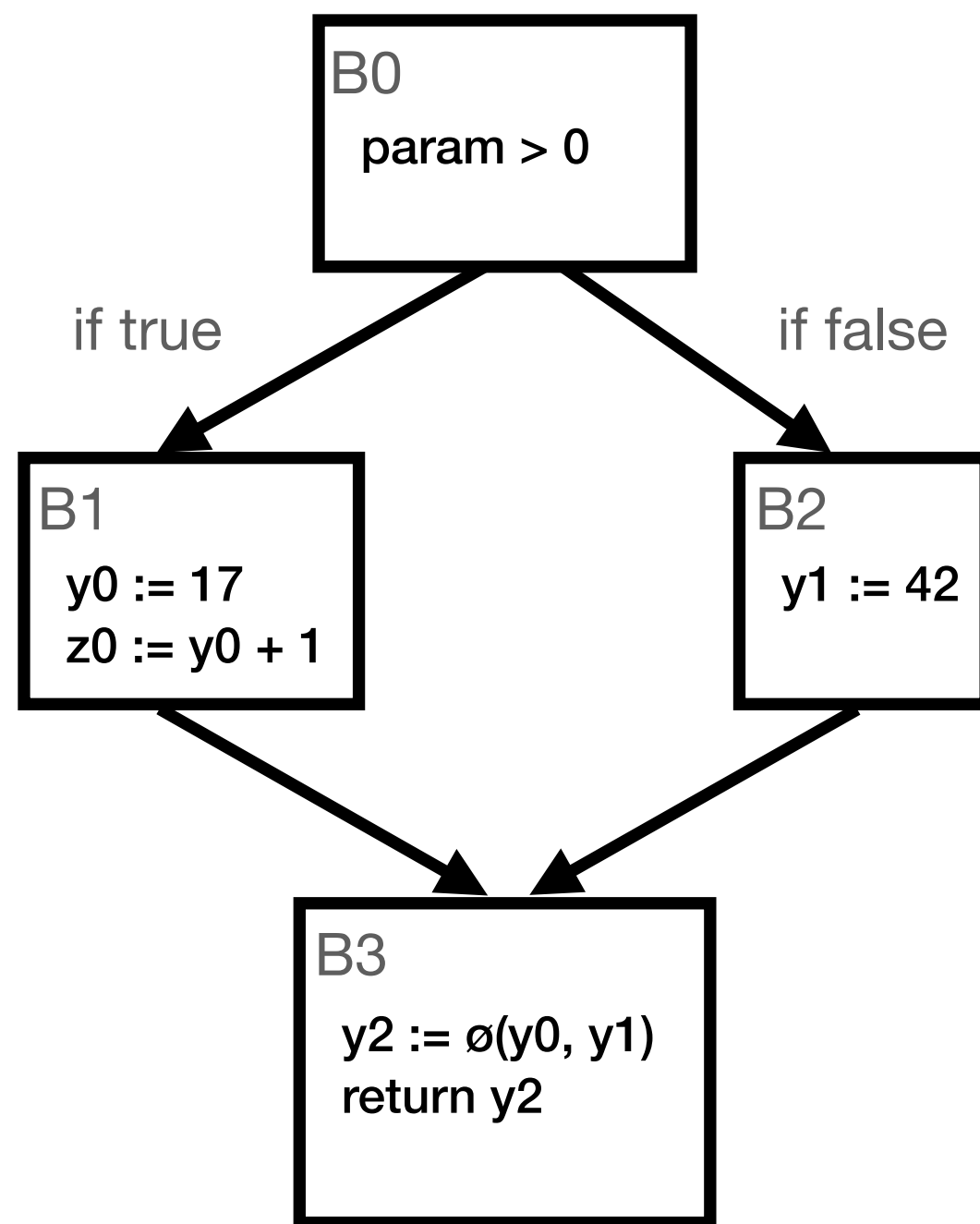


More complex dataflow analyses

- Local analysis per block (easy, no control flow)
 - Each analysis takes the block itself
 - and the incoming information from other blocks
- Propagate information forward/backwards depending on the analysis



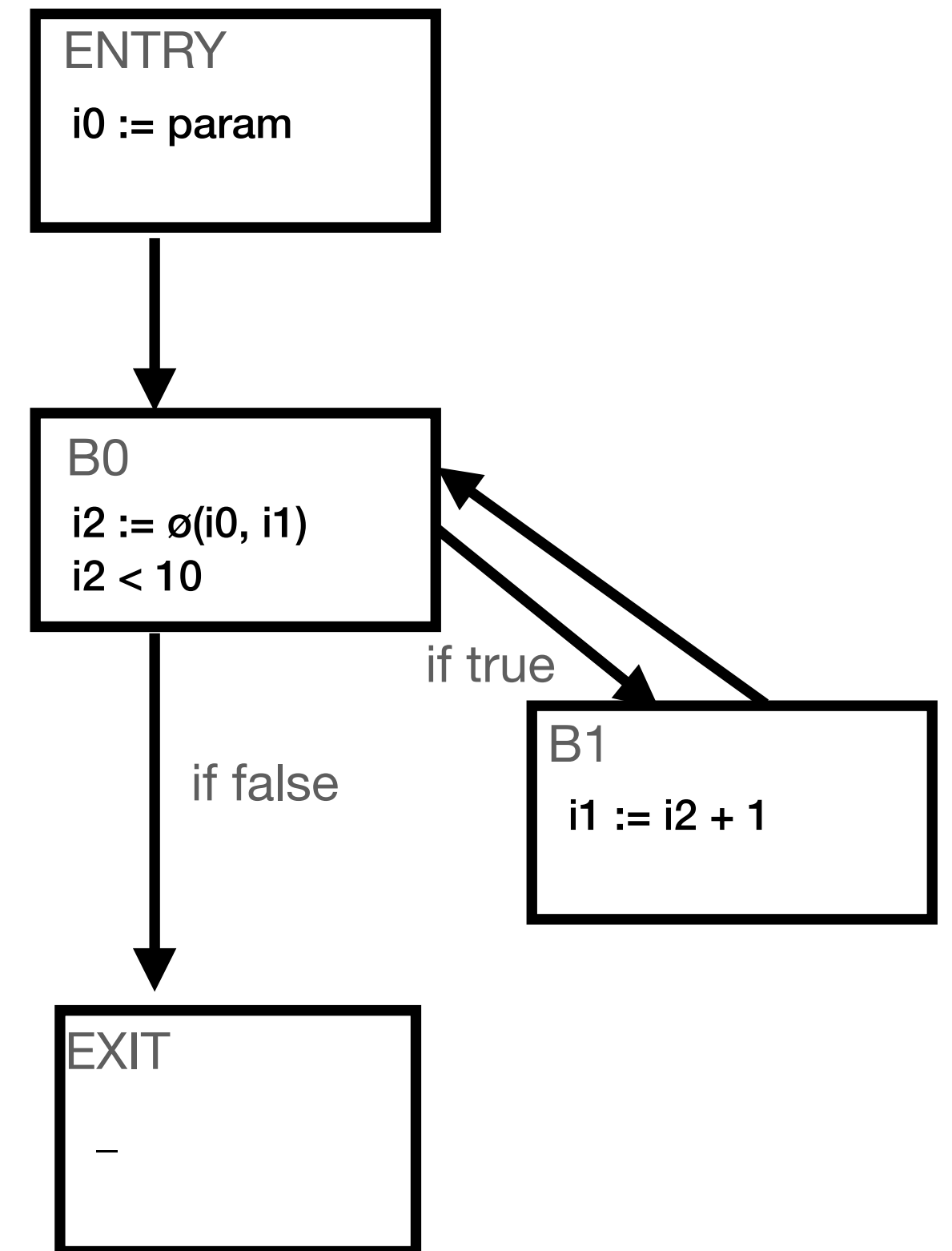
Example: forward constant propagation



- Can we propagate the values of y down?
- When should we analyze B3? reverse-post-order
- After B0: we know nothing about constants
- After B1: we know $y_0=17$ and $Z_0=18$
- After B2: we know $y_1=42$
- After B3: we know y_2 is not constant!

Loops?

```
def f(param){  
    local i := param.  
    [ i < 10 ] whileTrue: [  
        i := i + 1.  
    ]  
}
```



Building SSA in the standard way

- Start from non-ssa
- SSA is a “rename” refactor
- Each assignment $\Rightarrow$ *new unique* name
- What read corresponds to what write
 - Using a dominator tree

Efficiently Computing Static Single Assignment Form and the Control Dependence Graph

RON CYTRON, JEANNE FERRANTE, BARRY K. ROSEN, and
MARK N. WEGMAN

IBM Research Division

and

F. KENNETH ZADECK

Brown University

In optimizing compilers, data structure choices directly influence the power and efficiency of practical program optimization. A poor choice of data structure can inhibit optimization or slow compilation to the point that advanced optimization features become undesirable. Recently, static single assignment form and the control dependence graph have been proposed to represent data flow and control flow properties of programs. Each of these previously unrelated techniques lends efficiency and power to a useful class of program optimizations. Although both of these structures are attractive, the difficulty of their construction and their potential size have discouraged their use. We present new algorithms that efficiently compute these data structures for arbitrary control flow graphs. The algorithms use *dominance frontiers*, a new concept that may have other applications. We also give analytical and experimental evidence that all of these data structures are usually linear in the size of the original program. This paper thus presents strong evidence that these structures can be of practical use in optimization.

Categories and Subject Descriptors: D.3.3 [Programming Languages]: Language Constructs—*control structures; data types and structures; procedures, functions and subroutines*; D.3.4 [Programming Languages]: Processors—*compilers; optimization*; I.1.2 [Algebraic

Takeaways

- Code analysis can be organized as a dataflow analysis
- Iterative and worklist algorithms look for a *fixpoint*
- SSA code representation makes explicit use/def relations

Material

- Books
 - Dragon Book
 - Advanced Compiler Design & Impl
 - Engineering a Compiler

